WALCHAND COLLEGE OF ENGINEERING, SANGLI

(Government Aided Autonomous Institute)

WALCHAND LINUX USERS' GROUP

WLUG

Community | Knowledge | Share

METAMORPHOSIS

LIMITED SEATS

SAILING THROUGH THE CONTAINERS ERA

EXCITING PRIZES

11 & 12 APRIL

- DOCKER DRIFT
- DOCKER DEEPDIVE

- GO KICKSTART
- GO UNLEASHED

REGISTER HERE

SPONSORED BY

FOSS UNITED

CONNECT WITH US

₹349 | ₹649 | ₹899

MAIN & MINI CCF

FOR ANY QUERY
9579047160
7219685991

Ms. Pooja Bhosale President Walchand Linux Users' Group	Dr. A. J. Umbarkar Staff Advisor Walchand Linux Users' Group	Dr. R. R. Rathod HoD Information Technology	Dr. A. R. Surve Staff Advisor and HoD Computer Science and Engineering	Dr. M. M. Khot Dean (SW)	Dr. S. S. Rathod Director

TABLE OF CONTENT

- Goroutines
- Channels
- APIs
- Standard Libraries
- Json Encoding and Decoding
- Http Server

GOROUTINES

- Used to handle concurrent Programmes
- Goroutines can be thought of as lightweight threads
- The cost of creating a Goroutine is tiny when compared to a thread. Hence Go applications mainly have thousands of Goroutines running concurrently

GOROUTINES

Syntax:

- Put a keyword “go” before calling the name of Function

GOROUTINES

WITH goroutines:Runs at the same time

```go
func task1() {
    fmt.Println("Task 1")
}
func task2() {
    fmt.Println("Task 2")
}

func main() {
    task1()
    task2()
}
/*Output:
Task 1
Task 2
```

GOROUTINES

WITH goroutines:Runs at the same time

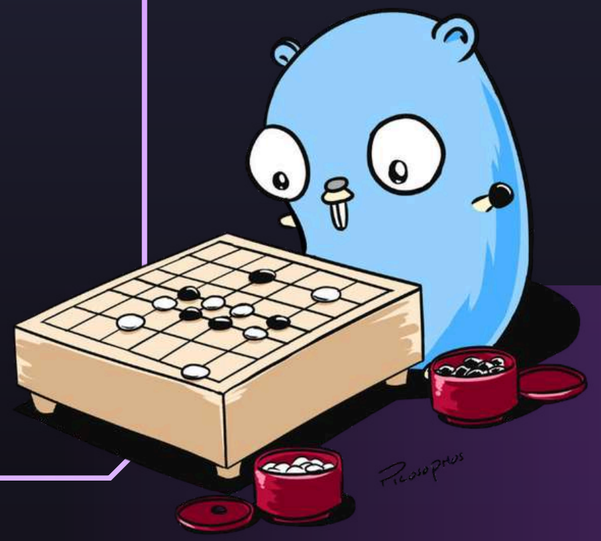
```go
func task1() {
    fmt.Println("Task 1")
}
func task2() {
    fmt.Println("Task 2")
}

func main() {
    task1()
    task2()
}
/*Output:
Task 1
Task 2
```

WLUG
COMMUNITY | KNOWLEDGE | SHARE

CHANNELS

CHANNELS

- Pipes using which Goroutines communicate
- Allows us to move data between different goroutines
- Do not communicate by sharing memory, instead share memory by communicating



SYNTAX

```go
chan T


a:=make(chan int)
```

channel of type T

Defines an int channel

SENDING, RECEIVING DATA

```
data := <- a // read from channel a

a <- data // write to channel a
```

- The direction of the arrow with respect to the channel specifies whether the data is sent or received

CHANNELS

- Sends and receives are blocking by default
- A blocking statement is a statement which blocks the execution of further lines of code after it

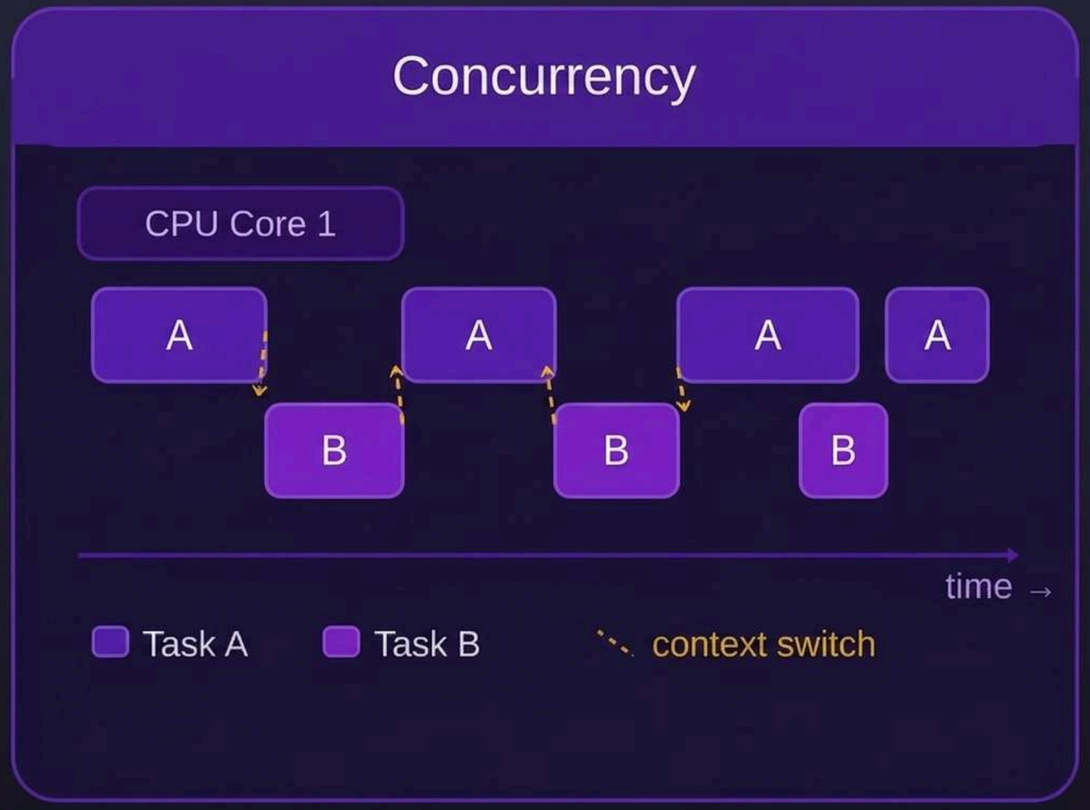
DEADLOCK

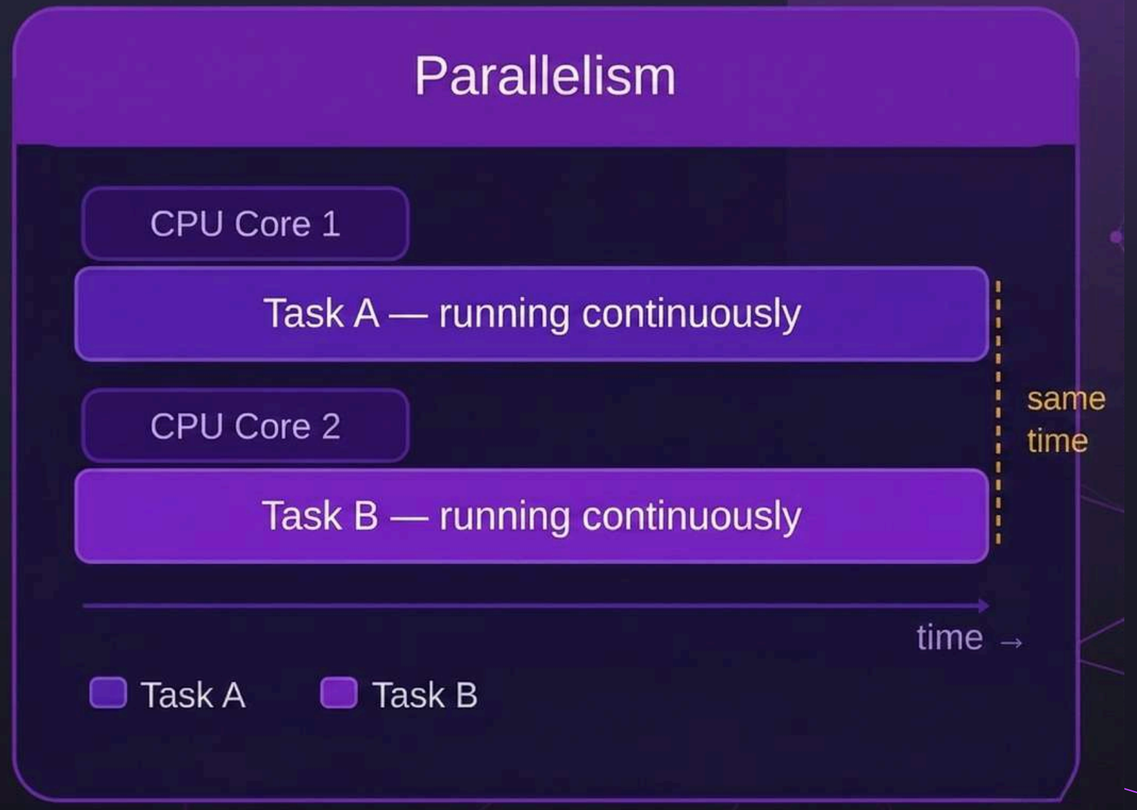
- If a Goroutine is sending data on a channel, then it is expected that some other Goroutine should be receiving the data

Interviewer
Explain deadlock and I'll hire you
Hire me and I'll explain it to you
Programmer

WLUG
COMMUNITY | KNOWLEDGE | SHARE

CONCURRENCY VS PARALLELISM


Concurrency
CPU Core 1
A
B
A
B
A
B
A
time →
Task A
Task B
context switch


Parallelism
CPU Core 1
Task A — running continuously
CPU Core 2
Task B — running continuously
same time
time →
Task A
Task B

CONCURRENCY

- Concurrency involves structuring a program so that two or more tasks may be in progress simultaneously

PARALLELISM

- Parallelism allows for two or more tasks to be executed simultaneously
- Parallelism requires more than one processor or thread, concurrency does not

APPLICATION PROGRAM INTERFACE

DEFINITION OF API

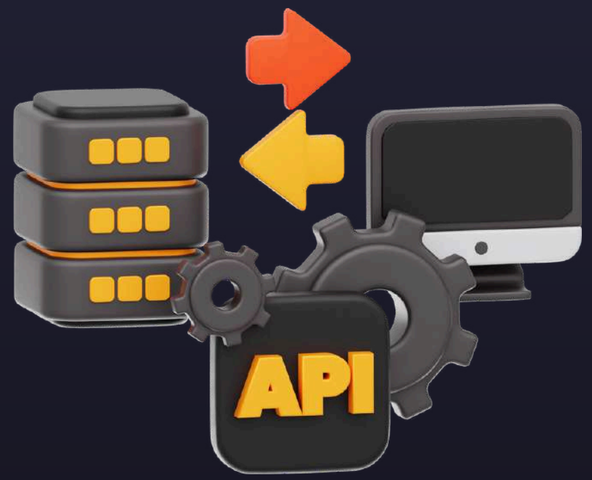
- Application Programming Interface (API) is a software interface that allows two applications to interact with each other

TYPES OF API

- Open APIs
- Internal APIs
- Partner APIs
- Composite APIs

API ARCHITECTURE

- REST
- SOAP
- Graph QL



HTTP METHODS

- GET
- PUT
- UPDATE
- DELETE

DATA FORMATS

- JSON
- XML
- Plain text

WLUG
COMMUNITY | KNOWLEDGE | SHARE

CLIENT
HOW DO I GET THIS DATA?
SERVER
THAT'S THE NEAT PART
405
You wouldn't get it

STANDARD LIBRARIES USED

FMT

Library	Overview
“fmt”	Package fmt implements formatted I/O with functions analogous to C's printf and scanf.

NET/HTTP

Library	Overview
"net/http"	Package http provides HTTP client and server implementations

TIME

Library	Overview
“time”	Package time provides functionality for measuring and displaying time

ENCODING/JSON

Library	Overview
"encoding/json"	Package json implements encoding and decoding of JSON

WLUG
COMMUNITY | KNOWLEDGE | SHARE

JSON ENCODING

JSON ENCODING

- Most web APIs return JSON responses
- encoding/json package
- NewEncoder
- Marshalling
- json tags

WHAT IS MARSHALLING?

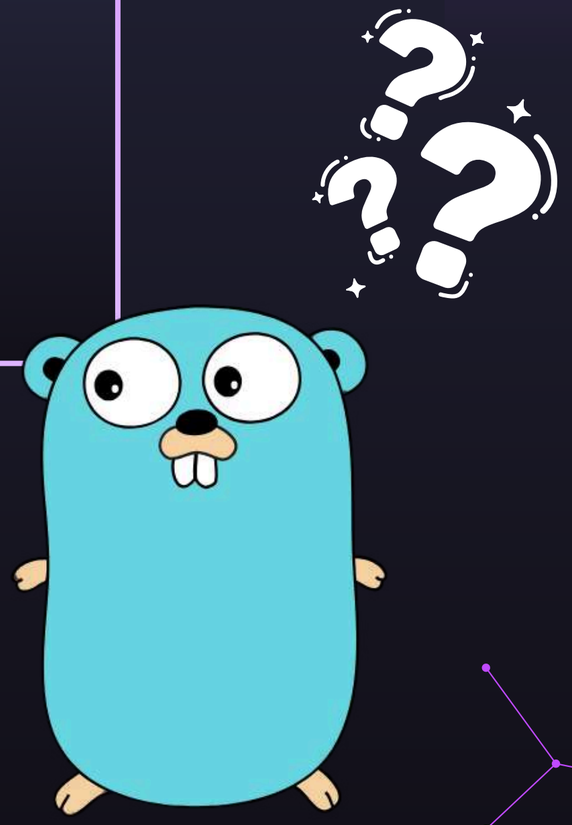
- Converting Go struct → JSON
- Key Function : json.Marshal()

WHAT IS NEWENCODER?

- Used for streaming JSON directly to output
- Common in HTTP responses
- Key Function : json.NewEncoder()

WHY USE NEWENCODER?

- Avoids intermediate byte array
- More efficient for large data
- Cleaner for web servers

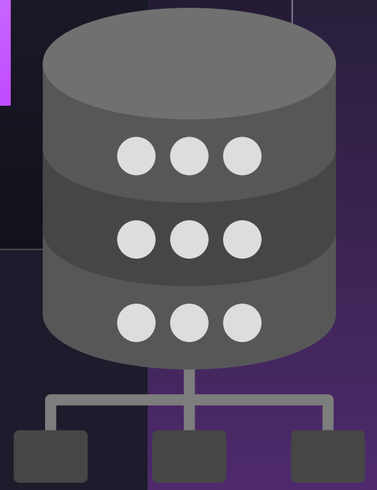


WLUG
COMMUNITY | KNOWLEDGE | SHARE

HTTP SERVER

WLUG
COMMUNITY | KNOWLEDGE | SHARE

WHAT IS SERVER?

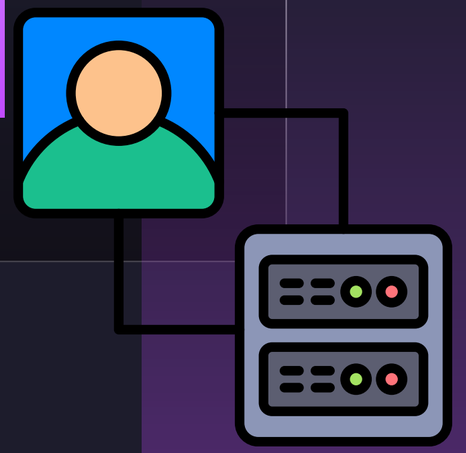


SERVER

- A server is simply a program or machine that provides services to other programs or users

WLUG
COMMUNITY | KNOWLEDGE | SHARE

WHAT IS CLIENT?



CLIENT

- Client is anything that sends a request to a server

HTTP

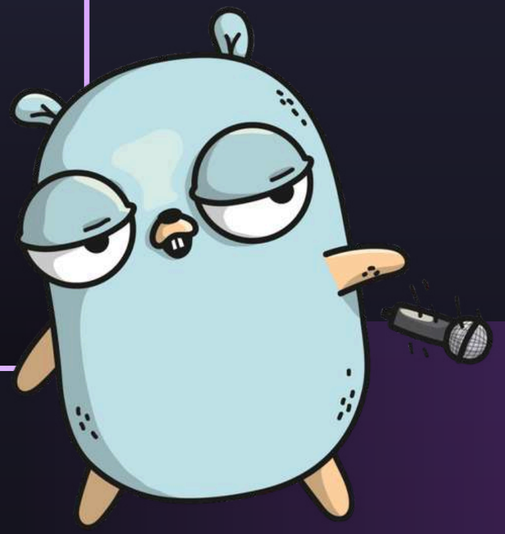
- HTTP stands for HyperText Transfer Protocol
- Request contains:
  - Method (GET, POST)
  - URL
  - Headers
  - Body
- Response contains:
  - Status code
  - Headers
  - Data

HTTP SERVER

- An HTTP server listens for requests from clients (like browsers) and sends responses.
- Common tasks:
  - Receive request from browser
  - Process request
  - Send response (HTML, text, JSON, etc.)

WHY GO FOR HTTP SERVERS?

- Built-in net/http package
- Very fast and lightweight
- Simple syntax
- Good for backend APIs and microservices



CREATING A BASIC HTTP SERVER

- Create handler function
- Register route
- Start server

NET/HTTP PACKAGE

- Go provides the net/http package to build web servers easily.
- Main components:
  - http.HandleFunc() → Register route
  - http.ListenAndServe() → Start server
  - http.ResponseWriter → Send response
  - http.Request → Access request data

HTTP METHOD

- HTTP methods define the type of action a client wants to perform on a resource.

GET	Retrieve data
POST	Create new data
PUT	Update existing data
DELETE	Remove data

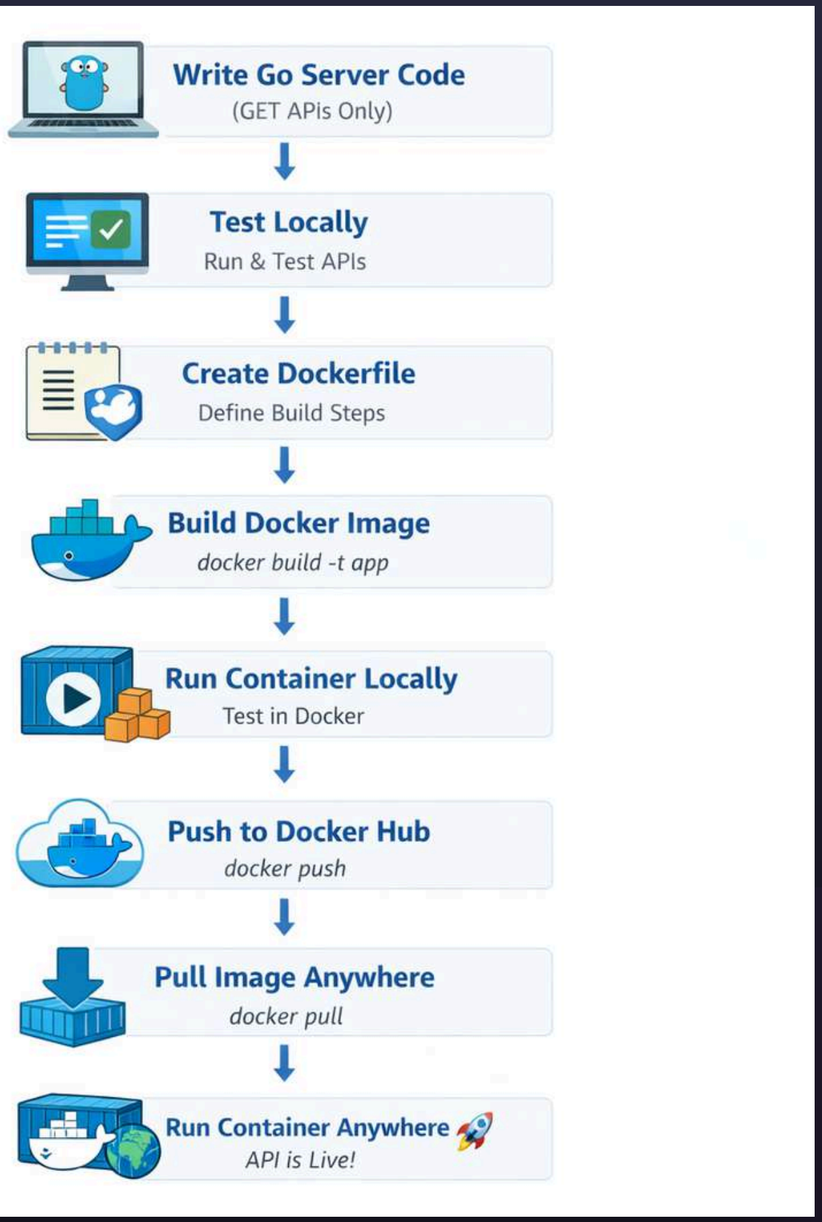
POSTMAN

- API testing tool
- Helps send HTTP requests easily
- No frontend required

POSTMAN

POSTMAN INSTALLATION

- Download Postman


Write Go Server Code
(GET APIs Only)
Test Locally
Run & Test APIs
Create Dockerfile
Define Build Steps
Build Docker Image
docker build -t app
Run Container Locally
Test in Docker
Push to Docker Hub
docker push
Pull Image Anywhere
docker pull
Run Container Anywhere
API is Live!

WLUG
COMMUNITY | KNOWLEDGE | SHARE

THANK YOU